Fondements de la programmation

Partiel 1, 31/11/2022

1 Spécification d'un fragment avec lists de OCaml

Il s'agit ici d'ajouter des listes, avec une liste vide [] et une opération $a :: l$ qui construit une liste en ajoutant un élément a dans une liste l

Nous utiliserons le langage :

TYPE

τ ::=	α		(Var. de type)
	`Num`		(tyNombres)
	$\tau_1 \to \tau_2$		(tyFonctions)
	τ `list`		(tyLists)

EXPRESSION

e ::=	x		(variable)
	n		(nombre littéral)
	$e_1 + e_2$		(addition)
	[]		(listeVide)
	e_1::e_2		(cons)
	$e_1\ e_2$		(application)
	`fun` $(x{:}\tau)$ $\Rightarrow$ e		(abstrTy)
	e_1 `where` $x = e_2$		(where)
	`let` x::y=e_1 `in` e_2		(défList)

CONTEXT $\Gamma ::= x_1{:}\tau_1, \ldots, x_n{:}\tau_n$

Priorité :

(défList) < (abstrTy)
< (where)
< (cons)
< (addition)
< (application)
< (TyFonctions)
< (TyLists)

Associativité gauche :
application, where, addition,

Associativité droite :
tyFonction, cons

Lieurs :
abstrTy : x dans e
where : x dans e_1
défList : x et y dans e_2

où les tokens $\alpha \in [']{[a-z]}$, $x \in [a-zA-Z_]{[a-zA-Z0-9_]}^*$, et $n \in [0-9]^+$ sont variables de types, variables de termes, et entiers, de OCaml.

1.1 Types

$$\frac{}{\Gamma, x : \tau \vdash x : \tau}\ (\text{id}) \qquad \frac{}{\Gamma \vdash n : \texttt{Num}}\ (\text{num}) \qquad \frac{}{\Gamma \vdash [] : \tau\ \texttt{list}}\ (\text{listeVide})$$

$$\frac{\Gamma \vdash e_1 : \texttt{Num} \qquad \Gamma \vdash e_2 : \texttt{Num}}{\Gamma \vdash e_1 + e_2 : \texttt{Num}}\ (+) \qquad \frac{\Gamma \vdash e_1 : \tau \qquad \Gamma \vdash e_2 : \tau\ \texttt{list}}{\Gamma \vdash e_1 :: e_2 : \tau\ \texttt{list}}\ (::)$$

$$\frac{\Gamma \vdash e_1 : \tau_1 \to \tau_2 \qquad \Gamma \vdash e_2 : \tau_1}{\Gamma \vdash e_1\ e_2 : \tau_2}\ (@) \qquad \frac{\Gamma, x : \tau_1 \vdash e : \tau_2}{\Gamma \vdash \texttt{fun}\ (x{:}\tau_1)\ \Rightarrow\ e : \tau_1 \to \tau_2}\ (\lambda\tau)$$

$$\frac{\Gamma, x : \tau_1 \vdash e_1 : \tau_2 \qquad \Gamma \vdash e_2 : \tau_1}{\Gamma \vdash e_1\ \texttt{where}\ x = e_2 : \tau_2}\ (\text{where}) \qquad \frac{\Gamma \vdash e_1 : \tau_1\ \texttt{list} \qquad \Gamma, x : \tau_1, y : \tau_1\ \texttt{list} \vdash e_2 : \tau_2}{\Gamma \vdash \texttt{let}\ x::y{=}e_1\ \texttt{in}\ e_2 : \tau_2}\ (\text{letList})$$

1.2 Système de transition

$$\frac{n_1 + n_2 = n}{n_1 + n_2 \mapsto n}\ (+) \qquad \frac{e_1 \mapsto e_1'}{e_1 + e_2 \mapsto e_1' + e_2}\ (+\text{G}) \qquad \frac{e_2 \mapsto e_2'}{e_1 + e_2 \mapsto e_1 + e_2'}\ (+\text{D})$$

$$\frac{e_1 \mapsto e_1'}{e_1\ e_2 \mapsto e_1'\ e_2}\ (@\text{G}) \qquad \frac{e_2 \mapsto e_2'}{e_1\ e_2 \mapsto e_1\ e_2'}\ (@\text{D}) \qquad \frac{}{(\texttt{fun}\ (x{:}\tau)\ \Rightarrow e_1)\ e_2 \mapsto e_1[e_2/x]}\ (\beta)$$

$$\frac{e_2 \mapsto e_2'}{e_1\ \texttt{where}\ x = e_2 \mapsto e_1\ \texttt{where}\ x = e_2'}\ (\text{whereD}) \qquad \frac{}{e_1\ \texttt{where}\ x = e_2 \mapsto e_1[e_2/x]}\ (\text{whereG})$$

$$\frac{e_1 \mapsto e_1'}{\texttt{let}\ x{::}y{=}e_1\ \texttt{in}\ e_2 \mapsto \texttt{let}\ x{::}y{=}e_1'\ \texttt{in}\ e_2}\ (\text{letLG}) \qquad \frac{}{\texttt{let}\ x{::}y{=}e_1{::}e_2\ \texttt{in}\ e_3 \mapsto e_3[e_1/x, e_2/y]}\ (\text{letLD})$$

1.3 Valeur

On appelle valeur uniquement :
- un entier n
- une fonction $\texttt{fun}\ (x{:}\tau)\ \Rightarrow e$
- une liste $v_1{::}\cdots{::}v_n{::}\ []$ contenant des valeurs $v_1, ..., v_n$

2 Questions

2.1 Arbre abstrait avec lieurs

Question A. ABT. Donnez les ABT des expressions suivantes :
- `let x::y=u::toto+foo::v in x y::[]`
- `(fun (bar:'a list->'a->'a list) => bar (a::[])) fun (x:'a list)=>fun (y:'a) => (x::y)`

Question B. alpha-equivalence. Donnez une autre représentation α équivalente de l'expression : `(fun (x:Num) => x+x where x=x+1)(3)+x`

Question C. substitution. Effectuez les deux substitutions suivantes
- `((fun (x:Num) => x+x where x=x+1)(3)+x)[42/x]`
- `((fun (y:Num) => x+x where x=x+1)(3)+x)[3*y/x]`

2.2 Types

Question D. Typage. Lesquels des expressions suivantes sont typables ? Donnez une dérivation ou une décoration de l'ABT quand c'est le cas, expliquez en une ligne le soucis lorsque ce n'est pas le cas
- `let x::y = [] in x (3::y)`
- `let x::y = [] in x 3`

Question E. Unicité. Prouvez qu'il n'y a pas unicité du typage

2.3 Réductions

Question F. Réduire le terme suivant. .

`(fun (l:(Num->Num) list) => let x::y = l in x 3) (f::f::[]) where f = fun (x:Num)=> x+x`

2.4 Preuves

Deux des 4 théorèmes suivant requièrent une induction, pour ceux-ci, faites les cas (id), (letList), ($\lambda\tau$), ainsi qu'un autre cas de votre choix.

Prouver 3 des 4 théorèmes sufit à avoir tous les points (le 4ième apporte un bonus).

Question G. Progression. Prouvez que ce language ne vérifie pas le théorème de progression, c'est à dire qu'il existe une expression e, close et typable, qui n'est pas une valeur mais ne peut pas se réduire.

Question H. Montrer la sûreté du type. c'est à dire que si $e \mapsto e'$ et $\Gamma \vdash e : \tau$ alors $\Gamma \vdash e' : \tau$

Question I. Prouvez que l'inverse n'est pas vrai. .

c'est à dire qu'il existe $e \mapsto e'$ tel que $\Gamma \vdash e' : \tau$ mais $\Gamma \not\vdash e : \tau$

Question J. Universalité des variables de types. .

Montrez que si $\Gamma \vdash e : \tau_1$, alors $\Gamma[\tau_2/\alpha] \vdash e[\tau_2/\alpha] : \tau_1[\tau_2/\alpha]$